

Career Compass — Technical Audit

Scope: Code quality, TypeScript hygiene, dead code, error handling, database schema, indexes, query patterns, RLS performance, bundle size, render performance, DevOps gaps, test coverage.

Standards referenced: React 18 best practices, Supabase performance recommendations, Web Vitals (LCP/INP/CLS), TypeScript strict mode.

Findings Summary

Severity	Count	Theme
● P0	4	No tests on critical scoring code · production console.logs · missing indexes on hot FKs · stale-schema-cache firefighting
● P1	9	TypeScript strictness gaps · dependency bloat (@anthropic-ai/sdk unused) · 926-LOC component · no code-splitting · no CI · no error monitoring · 14 any escapes · query waterfalls · duplicated stream-color tokens
● P2	10	Bundle size budget · image strategy · React Query underused · component re-renders · loading-state UX · React 18 StrictMode workaround pattern · etc.

Section A — Code Quality

A.1 ● P0 — Zero automated tests

Repo has:

- No *.test.ts / *.test.tsx / *.spec.* files anywhere.
- No vitest.config.* / jest.config.*.
- No playwright/ , no cypress/ .
- package.json scripts has no test entry.

The single most critical block of code in this product is src/lib/scoring.ts (calculateScores , getStreamRecommendation). It is **53 lines of arithmetic that decide what 14-year-olds study for the next 4 years** — and it has never been tested. Any future refactor of the weights, the Likert parser, or the personality bucketing can change a student's recommendation without anyone noticing.

Fix (this week):

```
npm i -D vitest @vitest/coverage-v8
```

Add to package.json :

```
"test": "vitest",
"test:coverage": "vitest --coverage"
```

Write the first 12 tests:

1. Pure-numerical student → Science gets > Commerce, Humanities.
2. Pure-business student → Commerce wins.
3. Pure-Humanities student → Humanities wins.
4. All-zero responses → all scores = 0; recommendation defaults predictably.
5. Skipped questions (missing keys) → handled without throwing.
6. Out-of-range Likert ('5' , '-1') → clamped or rejected, not silently scored.

7. Empty `aptMap` → scoring returns finite numbers (no NaN).
8. Boundary: `Science=70`, `Commerce=70` → tied; deterministic tie-break documented.
9. Personality map with unknown stream tag → ignored, not silently mis-attributed.
10. Mixed grade-prefix IDs (`apt_d_*` + `apt_*`) → grade detection picks deterministically.
11. `getStreamRecommendation` returns one of the three streams.
12. `calculateScores` is **pure** (same inputs → same outputs; no `Date/Math.random` in path).

Target: 90%+ coverage on `src/lib/scoring.ts` before any other test.

A.2 ● P0 — Production `console.log` statements

14 statements (see Security Audit A.4). Beyond the privacy issue, this is also a **performance regression** — console operations in tight loops slow down render and can hold references that prevent GC. Strip via Vite's `terser` `drop_console: true` in `vite.config.ts`:

```
build: {
  minify: 'terser',
  terserOptions: {
    compress: { drop_console: true, drop_debugger: true }
  }
}
```

A.3 ● P1 — 14 escapes to `any` in core types

`grep -n ": any\| as any" src/**/*.tsx src/**/*.ts` finds ~14 occurrences, concentrated in `Results.tsx` and `Dashboard.tsx`. Examples:

- `(allAssessments || []).map((a: { id: string }) => a.id)` — inlined shape literal instead of using the generated Supabase types.
- `(careers as { title: string; description: string }[])` — re-cast on read.

Fix: Generate Supabase TypeScript types and import them everywhere:

```
npx supabase gen types typescript --project-id <id> > src/types/database.ts
```

Then `import { Database } from '@types/database'` and use `Database['public']['Tables']['results']['Row']` directly.

A.4 ● P1 — `Results.tsx` is 926 lines

Single component handles: data loading, recalculation, AI narrative, PDF download, share, retake-modal, exam roadmap (153 lines of hard-coded constants alone), score-band rendering, stream-preference alignment, attempt history hooks. This violates single-responsibility, makes the file un-testable, and forces a 35KB+ component into the same bundle as the landing page.

Refactor target:

- `src/data/examRoadmap.ts` — extract the `EXAM_ROADMAP` constant.
- `src/components/results/StreamHero.tsx` — top banner.
- `src/components/results/ScoreBreakdown.tsx` — bar chart with deltas.
- `src/components/results/CareerList.tsx`
- `src/components/results/ExamRoadmap.tsx`
- `src/components/results/AIcounsellorNote.tsx`
- `src/components/results/RetakeModal.tsx`
- `src/hooks/useResults.ts` — all loading + calculation logic.

Target: `Results.tsx` < 200 lines after refactor.

A.5 🟡 P1 — Triple-fallback INSERT in `loadResults`

`Results.tsx:413–431` : tries insert with `attempt_number` → on stale-schema-cache error tries again without it → re-loads on either path. The comment says PostgREST schema cache may not have reloaded after migration 004. The proper fix is:

1. After running migration 004, **explicitly reload the cache** via Supabase Studio (Database → Schema → "Reload schema cache") or hit the PostgREST `/postgrest/reload` endpoint.
2. Remove the fallback path.

Carrying defensive fallbacks for migrations that are *years old* is a smell — pay it down.

A.6 🟡 P1 — Duplicated `STREAM_COLORS` / theme tokens

`STREAM_COLORS` is defined in:

- `src/pages/Results.tsx:13–17`
- `src/pages/Dashboard.tsx:17–30` (as `STREAM_CONFIG`)
- `src/pages/Careers.tsx:41–45`

Three files, three slightly different keys (`bg/light/border` vs `light/border/text/bar/dot` vs `bg/text/border/badge/icon`). A theme change requires 3 edits and *will* drift.

Fix: `src/lib/streamTheme.ts` :

```
export const streamTheme = {
  Science: { bg: 'bg-blue-600', light: 'bg-blue-50', border: 'border-blue-200', text: 'text-blue-700', bar: 'bg-blue-500', dot: 'bg-blue-600', badge: 'bg-blue-100 text-blue-700', emoji: '🔬' },
  Commerce: { bg: 'bg-green-600', light: 'bg-green-50', border: 'border-green-200', text: 'text-green-700', bar: 'bg-green-500', dot: 'bg-green-600', badge: 'bg-green-100 text-green-700', emoji: '💰' },
  Humanities: { bg: 'bg-indigo-600', light: 'bg-indigo-50', border: 'border-indigo-200', text: 'text-indigo-700', bar: 'bg-indigo-500', dot: 'bg-indigo-600', badge: 'bg-indigo-100 text-indigo-700', emoji: '📖' },
} as const
```

A.7 🟡 P1 — Bundled-but-unused `@anthropic-ai/sdk`

`package.json` declares `@anthropic-ai/sdk: ^0.78.0` as a dependency, but the code calls Anthropic's REST API directly via `fetch()`. The SDK adds ~80KB gzipped to the client bundle for nothing.

Fix: `npm uninstall @anthropic-ai/sdk`. After moving the AI call to a Supabase Edge Function (Security Audit A.1), import the SDK there if needed; that's a server-side Deno bundle and won't affect client size.

A.8 🟡 P1 — Two careers data sources

(Cross-ref Product Audit 1.5.) `src/lib/scoring.ts:topCareers` (5/stream, short) and `src/pages/Careers.tsx:ALL_CAREERS` (18 total, with salary/skills/industry) coexist. Consolidate to a single `src/data/careers.ts` source. The Results page can slice the 5 most-aligned for the recommendation; Careers page reads the whole list.

A.9 🟢 Other code-quality observations

- `src/lib/utils.ts` is **6 lines** — likely only contains `cn()`. Fine.
- `vite.config.ts` is 7 lines — minimum viable. Add `manualChunks` (see Section C).
- No ESLint rules beyond `eslint-plugin-react-hooks` and `eslint-plugin-react-refresh`. Add `@typescript-eslint/no-explicit-any: error` after fixing A.3.
- `tsconfig.json` should be inspected for `strict: true`, `noUncheckedIndexedAccess: true`, `noImplicitOverride: true`.

- Magic numbers in `scoring.ts` (40, 40, 20) and in the assessment engine (`minComplete: 20, 24, 12`) should be named constants in `src/lib/config.ts` .

Section B — Database & Schema

B.1 ● P0 — Missing indexes on hot foreign keys

All five user-data tables have FKs that the app filters on **every read**, but no indexes are declared in `001_initial_schema.sql` :

Table	FK column	Queries that filter on it	Index?
profiles	id (PK)	always	✓ (PK)
assessments	student_id	every dashboard / results load	✗
assessment_responses	assessment_id	every answer save + every score load	✗
results	student_id	every dashboard / results load	✗
parent_student_links	parent_id, student_id	every parent fetch	✗

At 100 users the lack of indexes is invisible. At 10,000 students with retake history, `select * from assessment_responses where assessment_id = '...'` becomes a full scan — and the app does this multiple times per Results-page load. Supabase auto-creates an index on the *primary* key but not on FKs.

Fix (new migration 010):

```
create index if not exists idx_assessments_student_id on public.assessments(student_id);
create index if not exists idx_assessment_responses_aid on
public.assessment_responses(assessment_id);
create index if not exists idx_results_student_id_created_at on public.results(student_id,
created_at desc);
create index if not exists idx_parent_student_links_parent on
public.parent_student_links(parent_id);
create index if not exists idx_parent_student_links_student on
public.parent_student_links(student_id);
create index if not exists idx_profiles_role on public.profiles(role) where role <>
'student';
create index if not exists idx_results_share_token on public.results(share_token) where
share_token is not null;
```

The last one is particularly important: every anonymous `/share/:token` page hit does `select * from results where share_token = ?` . Without an index this is a full scan over every result ever computed.

B.2 ● P1 — school_name is unconstrained text

(Cross-ref Product Audit 1.8.) Free-text → "DPS RK Puram" vs "D.P.S R.K Puram" → counsellor groupings break. **Fix:** introduce a `schools` reference table:

```
create table public.schools (
  id          uuid primary key default gen_random_uuid(),
  name        text not null,
  board        text check (board in ('CBSE', 'ICSE', 'State', 'IB', 'IGCSE')),
  city         text not null,
  state        text,
  unique (name, city)
```

```
);

alter table public.profiles
  add column school_id uuid references public.schools(id);
```

Seed with 5,000–10,000 known CBSE/ICSE schools (free dataset from UDISE+).

B.3 ● P1 — No soft-delete / no deleted_at

Every table is hard-delete-only. A student who deletes their account (when you build that flow per Security B.4) cascades-deletes their assessments, responses, and results, but you lose **aggregate analytics data forever** (e.g., "completion rate by school"). Adding a `deleted_at timestampz` column + filtering RLS on `deleted_at is null` preserves anonymisable rows for stats.

B.4 ● P1 — assessments UNIQUE constraint causes retake clutter

```
unique(student_id, type)
```

This forces each student to have **at most one** aptitude assessment row. On retake, the existing row is *re-used* (status flipped back to `in_progress`). But the migration history (007) and Dashboard dedup logic (`Dashboard.tsx:67–76`) suggest this constraint has been worked around in practice — the codebase fetches "ALL completed assessments... for each type" (`Results.tsx:279–283`) and picks the best one.

Either:

- (a) **Drop the UNIQUE** and let multiple rows exist per student/type, simplifying retake logic.
- (b) **Keep UNIQUE** but properly cascade-clear responses on retake (already done in `handleRetake`), so the single row truly represents the current attempt.

Pick one and remove the dead workarounds. The current state is the worst of both — a UNIQUE plus code that defeats it.

B.5 ● P2 — RLS policies use auth.uid() in JOINS without caching

Several policies call `auth.uid()` *inside* an `exists(...)` subquery. For each row, PostgreSQL re-evaluates `auth.uid()`. At scale, wrap in a sub-CTE or use `(select auth.uid())` for plan caching. See [Supabase RLS performance guide](#).

Example to refactor:

```
-- Before
using (auth.uid() = student_id or exists (
  select 1 from parent_student_links psl
  where psl.parent_id = auth.uid() and psl.student_id = results.student_id
))

-- After
using ((select auth.uid()) = student_id or exists (
  select 1 from parent_student_links psl
  where psl.parent_id = (select auth.uid()) and psl.student_id = results.student_id
))
```

B.6 ● P2 — results.recommended_stream vs results.stream are both used

Migration history shows both columns coexist. `Results.tsx` writes both (`stream` and `recommended_stream`). Dashboard reads `recommended_stream`. Pick one canonical column name and drop/rename the other.

B.7 ● P2 — Backups untested

You're on Supabase managed Postgres, which runs daily backups — but it's worth **actually restoring one to a fresh project** once per quarter and confirming the data is intact. "We have backups" is a *belief* until tested.

Section C — Performance

C.1 ● P1 — Whole app ships as one bundle

`vite.config.ts` has no `manualChunks` / no `import()` for routes. Every visitor downloads:

- 100KB+ of React + ReactDOM.
- 50KB+ of `@radix-ui` components (dialog/dropdown/progress/toast/separator).
- 80KB+ of `@anthropic-ai/sdk` (unused — see A.7).
- ~80KB of `jspdf` .
- 40KB+ of `@tanstack/react-query` .
- ~50KB of `lucide-react` icons (if not tree-shaken — verify).
- All page components — even unauthenticated visitors download `AdminDashboard.tsx` (281 LOC) and `CounsellorDashboard.tsx` (499 LOC).

Estimated initial JS payload: ~**500KB gzipped**. On a 3G connection (still common in tier-2/3 India), that's ~**5 seconds** of download before the page is interactive.

Fix: convert routes to lazy in `src/App.tsx` :

```
const Dashboard = lazy(() => import('@pages/Dashboard'))
const Results = lazy(() => import('@pages/Results'))
const Careers = lazy(() => import('@pages/Careers'))
const AdminDashboard = lazy(() => import('@pages/AdminDashboard'))
const CounsellorDashboard = lazy(() => import('@pages/CounsellorDashboard'))
```

And vendor-chunk in `vite.config.ts` :

```
build: {
  rollupOptions: {
    output: {
      manualChunks: {
        vendor: ['react', 'react-dom', 'react-router-dom'],
        supabase: ['@supabase/supabase-js'],
        radix: ['@radix-ui/react-dialog', '@radix-ui/react-dropdown-menu', ...],
        pdf: ['jspdf'],
      }
    }
  }
}
```

Target: **landing-page route loads in < 150KB** gzipped of JS.

C.2 ● P1 — Query waterfall on Results page

`Results.tsx:loadResults` :

1. `Promise.all([fetch aptitude, fetch interest, fetch personality])` — good (parallel).
2. **Then** `select * from results ... limit 2` — sequential.
3. **Then** `findBestAssessmentId(...)` which is a `for` loop with `await` per assessment → **sequential** queries against `assessment_responses` .
4. **Then** another lookup of `assessment_responses` to build the map.
5. **Then** `INSERT` to `results` .

6. **Then** UPDATE to `results` (for AI narrative).

On the worst path that's ~**8 sequential round-trips to Supabase**. Each ~80-150ms in tier-2 India. **Total: 800-1200ms+ on the critical results-rendering path.**

Fix:

- Combine steps 3+4 into a single query: `select assessment_id, question_id, answer_value from assessment_responses where assessment_id in (<list>)`.
- Issue steps 2 and 3 in parallel.
- Move the AI narrative call to fire-and-forget (already partly done) but ensure it doesn't block the render.

Or, better: **create a Postgres function** `public.fetch_results_bundle(student_uuid uuid)` that does all the lookups server-side in one round-trip:

```
create or replace function public.fetch_results_bundle(student_uuid uuid)
returns json language sql security invoker stable as $$
select json_build_object(
  'aptitude_responses', (select json_agg(...)),
  'interest_responses', (select json_agg(...)),
  'personality_responses', (select json_agg(...)),
  'latest_result', (select to_json(r) from results r where ...)
);
$$;
```

One RPC call replaces 6 queries.

C.3 ● P1 — delete-then-insert per answer

`AssessmentEngine.saveAnswer` does `delete + insert` on every Likert click. On the 24-item interest test, that's **48 round-trips**. Use `upsert` instead:

```
await supabase
  .from('assessment_responses')
  .upsert(
    { assessment_id, question_id: questionId, answer_value: value },
    { onConflict: 'assessment_id,question_id' }
  )
```

One call instead of two. Also resilient if the user clicks twice rapidly.

C.4 ● P2 — No image strategy

The app uses emoji and icons (`lucide-react`) — no raster images. Good. But the moment a sample-report screenshot or a school logo is added to the landing page, you'll need WebP + AVIF + `<picture>` + width/height attributes to avoid CLS. Plan for it before shipping the marketing assets.

C.5 ● P2 — React Query installed but barely used

`package.json` has `@tanstack/react-query: ^5.90.21`, but components hand-roll their own `useState/useEffect/loading/error` patterns. React Query would give you free request deduplication, caching across page navigations, and stale-while-revalidate UX.

Fix: wrap auth + profile + results queries in `useQuery`. Single biggest win: the Dashboard fetches the same 3 assessments + results that the Assessment Engine and Results page also fetch. React Query would cache across all three pages.

C.6 ● P2 — `loadResults` runs on a guard-ref instead of React Query

The 80-line "concurrency guard" with `loadResultsRunning.current = true` (Results.tsx:202, 273) exists because React 18 StrictMode double-fires effects in development. **React Query's `useQuery` handles this natively.** Adopting React Query (C.5) makes the guard unnecessary.

C.7 ● P2 — No web-vitals telemetry

Without `web-vitals` reporting, you don't know your LCP, INP, CLS in production. Add:

```
import { onLCP, onINP, onCLS } from 'web-vitals'
onLCP(m => posthog.capture('webvital_lcp', { value: m.value }))
```

Targets: LCP < 2.5s, INP < 200ms, CLS < 0.1 — required for school procurement that runs `pagespeed.web.dev` over your URL.

Section D — DevOps, CI/CD, Monitoring

D.1 ● P1 — No CI

No `.github/workflows/` directory. Every push relies on the developer remembering to:

- Run `npm run lint`.
- Run `npm run build` (which is `tsc -b && vite build` — so TS errors block, good).
- Run tests (currently none).

Fix: `.github/workflows/ci.yml` :

```
name: CI
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npm run build
      - run: npm test # once tests exist
```

Plus Netlify deploy previews (built-in if the repo is connected).

D.2 ● P1 — No error monitoring (Sentry / Bugsnag / Rollbar)

Every error in production becomes invisible — users report it via WhatsApp at best, churn at worst. **Sentry free tier is 5K errors/month** — enough for early stage.

Fix:

```
import * as Sentry from '@sentry/react'
Sentry.init({
  dsn: import.meta.env.VITE_SENTRY_DSN,
  tracesSampleRate: 0.1,
  environment: import.meta.env.MODE,
})
```


Wrap in `<Sentry.ErrorBoundary>` at the top of `App.tsx`.

D.3 🟡 P1 — No analytics

(Cross-ref Product Audit cross-cutting.) Without PostHog/Plausible you have no funnel data. Free PostHog cloud + identify by Supabase user-id = end-to-end funnel in ~30 min.

D.4 🟡 P2 — Netlify config is barebones

`netlify.toml` has no `[[headers]]` (see Security A.8), no `[context.deploy-preview]` env split, no functions-bundling tuning. As soon as you add an Edge Function or an API proxy, this file needs expansion.

D.5 🟡 P2 — No environment-tier separation

There appears to be a single Supabase project (`ktkhidmpuejayabfnudi`). Production migrations land directly. **Fix:** create a `staging` Supabase project; run migrations there first; promote with `supabase db push`.

D.6 🟢 — Build script is correct

`"build": "tsc -b && vite build"` ensures TS errors fail the build. Good.

Section E — Verdict

The codebase is **above average for a solo-dev-built MVP**:

- TypeScript throughout.
- Server-side auth triggers.
- Idempotent migrations.
- React 18 patterns (refs for stale-closure issues, StrictMode-aware).
- Reasonable component decomposition (auth routes, layout, shared toast).

The **technical debt is concentrated in 4 places**:

1. **Zero tests** on the most consequential 50 lines of code.
2. **Production debug logs** that leak PII and slow renders.
3. **Missing indexes** on every foreign key the app filters on.
4. **One 926-line component** that owns half the product surface.

Closing the 4 P0 items is **3-5 engineering days**. Adding the 9 P1 items is another **2-3 weeks**. After that the platform is ready for a 10x scale-up without re-architecture.

Bottom line: *The platform's engineering quality is healthier than its measurement quality (psychometric audit) or its compliance quality (security audit). Fix the tests first, the indexes second, and the bundle-splitting third — the rest is hygiene.*